Team 4: Dataset agnostic quality assessment framework based on LLMs, semantic outlier detection and improved TFDV Schema Generator

Abdollahzadeh A., Afroozeh O., Dominici L., Liu H.J., Nguyen M. D.

ABSTRACT

The project aims at creating a tool to perform quality assessment and validation of datasets employed in ML pipelines. The tool is able to work with any dataset that has column names, without any background or domain knowledge. Throughout the pipeline, we are able to identify errors, represented by low quality data. Semantic errors are found via a semantic outlier detection system based on embeddings, while syntax errors are detected via an LLM based semantic type inference and subsequent RegEx validation. This approach is complemented with a specialised version of TensorFlow Data Validation Schema Generation module.

1 INTRODUCTION

The goal of the project is to develop a tool to quantify the data quality of datasets employed in ML pipelines. This task is extremely important due to the impact of training data on ML model performances [11]. The tool has been designed to work without human intervention. The pipeline that we have developed does not require supervision and it's supposed to work with diverse datasets, without relying on domain specific knowledge. We set these requirements in order to have a tool compatible with potentially any tabular dataset with column names. In order not to use metadata or background knowledge, we add an additional layer to the data validation pipeline: the metadata inference layer. During the metadata inference phase, we leverage everything we have (column names and values) to understand what the intended format and schema of the dataset is, being very careful not to over fit the data. We use two approaches to infer metadata: LLM-based semantic type inference and an improved version of TensorFlow Data Validation Schema Generator. After inferring the schema and the data types, we then move onto the error detection phase. During this phase, we actively look for syntactic and semantic errors in the dataset since these kind of errors lower the quality of the dataset. Semantic errors are spot by a semantic outlier detection algorithm based on value embeddings and clustering, allowing us to detect subtle errors that classic data linter would not be able to notice [13]. Syntax errors are identified through the usage of sets of Regex expressions, which are applied to the data based on the type of data that the LLM has inferred. The tool's performance have been evaluated with two distinct tests. The first "synthetic" test consists in adding sensible errors to the dataset and using our tool to spot and remove the errors. We evaluate which errors are spotted and how they affect relevant metrics. The second "natural" test consists in using AutoML to evaluate the performance of a dataset before and after the pipeline cleanup. The goal is to prove that our pipeline is able to identify low quality data that could negatively influence the outcome of subsequent ML pipelines.

The end goal of this tool in a usual ML pipeline is to spot and remove wrong or potentially misleading data from a training dataset, further increasing the ML model prediction accuracy as a consequence of improved data quality [6].

2 PROBLEM & APPROACH

Problem Statement. The goal of the project is to develop a tool capable of parsing and quantifying the data quality of a large number of datasets used in ML scenarios. With low-quality dataset, we mostly refer to dataset containing: missing values, outliers, typing error, format inconsistencies, duplicated entries or mislabeled training data. There are no detailed specs about the kind of dataset to use because it is intended to not have any.

Approach. The first and most important challenge in quantifying the dataset quality is to understand what is high quality (good) and bad quality (bad) data. The same piece of information could be right and coherent as well as being wrong and meaningless. Thus, most of it, depends on the context. As shown in 1 we can see that the same information, if meant as an email address is a high quality information, while under a different field, is clearly wrong and almost unusable.

Data	Column name	Column email
alan.turing@bletchpark.uk	Bad	Good
Alan Turing	Good	Bad

Table 1: Context dependent data correctness

Understanding the context, or the metadata of the table is key to understand what's a high quality data and what's not. Looking at the state of the art we've noticed that most tools rely on metadata, either provided by the user or inferred from the schema. Some of the tools we've analysed are TensorFlow Data Validation (TFDV) [2] , part of the TFX framework, and Deequ, a library to implement "unit test for data" [12] .

The first part of our pipeline is indeed the metadata inference phase. During this phase, we try to infer the intended schema and the metadata that could help discriminate with more accuracy among good or bad data. This phase is realized by using two different tools: LLM based semantic type inference and an improved version of TFDV Schema Generation. LLMs are used in order to take into consideration the column names as well as the column values. Our improved version of the TFDV Schema Generation instead, provides us with the intended schema for a dataset. Standard TFDV would return a schema that fits all the dataset values, being easily tricked into overgeneralizing some feature types because of few outliers in that column. TFDV also spots some major syntax errors.

The second part of the pipeline regards the error identification. Semantic errors are identified through a semantic outlier detector. It relies on embeddings and on clustering to spot textual outlier, numeric outlier are instead treated in a more standard mathematical fashion. Syntax errors are spot thanks to the findings of the metadata inference phase and the subsequent usage of sets of regex expression.

Implementation. All the code can be found in the project repository on GitHub [1].

2.1 LLM based semantic type inference

Our semantic type inference is based on 3 different LLMs: ChatGPT 3.5 accessed through APIs, Mixtral 8x7B [5] running locally on a Nvidia RTX4090 GPU server and LLaMa 2 7B [15] running locally on the same server. Our query to the LLM contains the column name, a limited number (<50) of sampled valued from that column, and a list of data types (DATE, TIME, DATETIME, URL, EMAIL, INT, FLOAT, JSON, BOOLEAN and others). Being the data types very intuitive to understand we didn't need to formally define them since the LLM can rely on its training data to understand what they mean. This behaviour allows the tool to be very flexible and capable of recognizing data in formats that we did not think about.

2.2 Improved TensorFlow Schema Generation

TFDV has originally been implemented for improving continuous ML pipelines in terms of robustness against data errors appearing in batches of new data. [2]. However, domain knowledge is needed to provide an initial schema and validate the schema generated from incoming batches. Without domain knowledge, TFDV infers the data type (String, Float or Integer) for each column and provides a domain of valid values for string columns. Applied on a whole dataset it is prone to overfitting, as every row is considered as true and correct. We introduce a batch-wise application of TFDV and a schema aggregation mechanism that generates a non overfitting schema dependent on three parameters: relative batch size, relative aggregation threshold and distinct value threshold. Using the aggregated schema, we can identify rows that have an unexpected and suspicious format.

Table 2 consists of example data and how it would be divided into batches with relative batch size 0.2. Applying TFDV on the whole dataset would lead to price being identified as String and the faulty value "big" being included in the domain for the size domain. The table also shows the inferred column types and domains by TFDV for the respective batches. Assuming an aggregation threshold of 0.8, regarding the type for price and domain for size, there are two types of batches with valid values that occur two times each (Price: Float, Integer; Size: S, XL, M, L). After merging two batches the threshold is reached (4/5 batches). Therefore, the aggregated schema consists of float as type for the price column and excludes the faulty value for the size column.

Price	Size	Type Price	Domain Price	Type Size	Domain Size	Batch
19 8	XL S	Integer	-	String	S, XL	#1
8.99 12	L M	Float	-	String	M, L	#2
5,99	big M	String	5,99	String	big, M	#3
4 7.50	XL S	Float	-	String	S, XL	#4
15 19	L M	Integer	-	String	M, L	#5

Table 2: Example dataset divided into batches with relative batch size of 0.2 and inferred schema for each batch by TFDV

2.3 Semantic outlier detector

Our approach to identify semantic outliers, is an embedding-based one. We utilize the capabilities of pre-trained language models, specifically, the `all-MiniLM-L6-v2` model, available on Hugging Face[2], which is a transformer architecture designed as a smaller and more efficient version of BERT, with the purpose of generating sentence embeddings. After generating embeddings for each unique value in the categorical column, we proceed to detect the outliers. For each unique value v_i, we calculate the 'mean vector' of the rest of the values.

The mean vector takes into consideration embeddings and their frequencies. After calculating the mean vector of other values, we compare it to the current value vector. If

[1] https://github.com/Leodom01/5284DAPR6Y-DataPreparation-Project

[2] https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

the Euclidean distance between them surpasses a certain threshold, we conclude that the semantic meaning of that value is out of context and report it as an outlier.

This outlier detection module performs effectively in most contexts, but it has its limitations. First, there's the lack of ground truth: without a predefined benchmark or labeled data indicating what counts an outlier, evaluating the accuracy of detected outliers is challenging.

Second, there is the Semantic Spread Issue, where fields with broad or diverse categories may lack sufficient semantic closeness. Lastly, the approach's effectiveness heavily relies on the embedding model chosen; certain models may not always capture the semantic similarity accurately, especially in nuanced contexts.

2.4 Regex syntax validator

We have developed a comprehensive repository of RegEx patterns tailored to validate syntax errors in datasets containing diverse data types commonly found in databases. By integrating this repository with potential regex outputs generated by LLM, we employ a systematic approach to identify syntax errors within the dataset. This method involves iteratively applying various regex patterns associated with each data type until the most compatible pattern is identified for validation. Entries failing to conform to the most compatible pattern are flagged as errors. This synergistic framework ensures robust error detection even in scenarios where the LLM output is invalid or inconclusive.

3 EVALUATION

Experimental Setup. We ran our experiments on a machine which runs on Linux Mint 21.3 x86_64, utilizing a 13th Gen Intel i7-13700K CPU clocked at 5.300GHz and an NVIDIA GeForce RTX 4090 GPU. Each experiments was run multiple times and an average result was reported.

Datasets. The dataset used for testing are Audible [3], CarPrices [1], Glassdoor [9], Kickstarter [8], Stack Overflow [14], and Wearables [7]. Datasets in the 100Mb range, only tabular with a blen of numeric and literal (short and long) values.

3.1 Outlier detection performance

One core component of our pipeline utilizes embeddings for identifying outliers within the data. Typically, generating these embeddings requires significant computation. To address this challenge with large datasets, we resort to a caching strategy where embeddings are computed solely for each distinct value in the column. Therefore, the computation time scales linearly with the number of unique values, as depicted in Figure 1. This experiment was run with different datasets with varying amount of unique values. Additionally, we conducted a further test on the outlier detection module

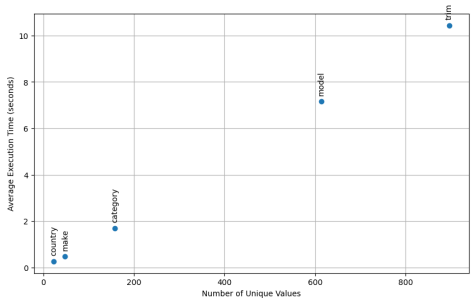

Figure 1: Average Execution Time vs Distinct values

to assess its performance. This test involved adding different types of outlier values into a categorical column to evaluate the capability of the outlier detection module. The detection rates of these outliers are presented in the table below.

Outlier Type	**Number of outliers**	**Detection Rate**
Arbitrary Anomalies	100	100%
Typographical Errors	100	67%
Contextual Outliers	100	83%

Table 3: Effectiveness of the Outlier Detection Module

3.2 LLM based syntactic type inference evaluation

Mean Type Accuracy	Sample Size			
Indipendent runs [Mean runtime]	5	**15**	25	35
1 [0.13s]	84.50%	**92.95%**	91.54%	87.32%
5 [0.11s]	90.71%	90.14%	91.54%	91.54%
10 [0.12s]	90.14%	91.54%	91.54%	91.54%

Table 4: Accuracy of Type prediction in ChatGPT3.5

Mean Regex Accuracy	Sample Size			
Indipendent runs [Mean runtime]	**5**	15	25	35
1 [1.48s]	28.57%	71.42%	57.14%	57.14%
5 [1.47s]	**85.71%**	85.71%	85.71%	85.71%
10 [1.78s]	85.71%	85.71%	85.71%	85.71%

Table 5: Accuracy of Regex prediction in ChatGPT3.5

The results shown in Tables 4 5 demonstrate that ChatGPT 3.5 consistently yields superior outcomes with minimal time consumption, whereas Mixtral achieves satisfactory results. Conversely, Llama 2 exhibits the poorest performance among the models assessed. Mixtral and Llama results have not been reported due to space constraints on the paper, the most relevant aspect is that the Mixtral runtime is tripled compared to ChatGPT, most likely due to the model being locally deployed. The next graphs represent the results of the LLM performances averaged on all datasets. The input to the LLM consists in: column name, samples (in variable sample size) of that column, list of types the LLM can pick from.

Executing the same input multiple times might not consistently produce identical outcomes. Running the input multiple times enables selecting the most frequently occurring result. Tables 6 and 7 show performances of the same experiment but with examples provided to the LLM. The examples are "EXAMPLE INPUT1: column name: Country; values: US, UK, US, CN, IT, NL, NL" to assist in improving its accuracy in determining the data type. No relevant improvement has been found.

Mean Type Accuracy	Sample Size			
Indipendent runs	5	15	25	35
1	85.91%	85.91%	87.32%	83.09%
5	85.915%	87.32%	85.91%	85.91%
10	87.323%	87.323%	**88.732%**	85.915%

Table 6: Accuracy of Type prediction in ChatGPT3.5 with examples

Mean Regex Accuracy	Sample Size			
Indipendent runs	5	15	25	35
1	42.85%	42.85%	57.14%	28.57%
5	71.42%	71.42%	71.42%	71.42%
10	71.42%	71.42%	71.42%	71.42%

Table 7: Accuracy of Regex prediction in ChatGPT3.5 with examples

3.3 Metrics

During our evaluation process, we apply data quality metrics proposed by Redyuk et al. in a column-wise manner[10]: **Completeness** quantifies the proportion of non-missing values, suggesting the dataset's readiness for analysis. **Approximate Count of Distinctive Values** employs the hyperloglog algorithm to estimate the number of unique values efficiently, reflecting data diversity. **Ratio of the Most Frequent Value** utilizes the count sketch approximation to determine the prevalence of the most common value, indicating potential data skewness. **Index of Peculiarity for Textual Data** evaluates the uniqueness of text trigrams against a reference table, assisting in the identification of typos and peculiarities in textual data.

3.4 Error generator evaluation

We evaluate our approach by manually adding errors of different types by our error generator at a fixed rate, i.e. 20%. In string columns, it imputes the dataset with typographical errors, in numerical columns, it adds outliers or placeholders. For special string formats, e.g. datetime, URL, email, it applies sensible changes to format and values. We apply the metrics on the datasets before and after imputing the errors as well as on the dataset without the detected rows by our service. We also measure how many rows containing errors were detected. Table 8 displays the metrics for a string column (category), special string column (deadline) and numeric column (usd). It shows that our data quality pipeline recovers most metrics. However, for distinct value metrics it reduces the number significantly as rare values are detected as anomalies.

Metric	Original dataset	Dataset with errors	Dataset with errors removed
category_Compl	1.00	0.80	1.00
category_CountDist	160	161	158
category_FreqRatio	0.06	0.20	0.05
category_PecyIndex	7.02	6.92	6.64
deadline_Compl	1.00	0.80	1.00
deadline_CountDist	3155	6193	3104
deadline_FreqRatio	0.00	0.20	0.00
deadline_PecIndex	6.99	6.87	7.13
usd_Compl	0.99	0.83	1.00
usd_CountDist	94678	142939	44825
usd_FreqtRatio	0.18	0.12	0.15

Table 8: Metrics for selected columns for kickstarter dataset

3.5 AutoML evaluation

Model	Accuracy Before	Accuracy After
KNeighborsUnif	11.78%	12.64%
KNeighborsDist	12.02%	12.88%
NeuralNetFastAI	36.85%	38.44%
RandomForestGini	31.73%	33.22%
RandomForestEntr	29.29%	30.82%

Table 9: AutoML results before and after cleanup

The second evaluation we have performed consists of running an AutoML pipeline on the original dataset and on the cleaned dataset. After all, the goal of quantifying data quality is to spot errors to remove and eventually improve the performances of any pipeline that used the cleaned dataset. In order to show that our tool is capable of enhanching end-to-end performances of ML pipelines we've used AutoGluon[4], an open-source AutoML library by Amazon. After employing AutoGluon to evaluate our dataset pre- and post-cleanup pipeline, we discovered that the cleaned data yielded higher accuracy. The dataset used in this case is the Audible dataset (uncleaned version).

4 DISCUSSION AND CONCLUSION

Our dataset quality assessment tool is a new and rather unique instrument for evaluation of datasets without any background knowledge or metadata information. Both our evaluations methods returned promising results. NLP technologies have proven to be good allies in automatically learning schemas and formats and should be further investigated. LLMs, adequately supervised, are a great tool in cases where the data content is unknown and unexpected.

The lack of ground truth is by far the biggest challenge in having a functional and useful tool. Making it hard to

discriminate among good and bad data, highlighting a crucial area for future improvements.

Further development would focus on making the system compatible with Apache Spark and low level libraries for optimized embedding computation. More precise outlier detection algos and more specific semantic data types could benefit the tool. More and more various datasets should be tested, both in terms of data diversity and also in terms of size.

Detailed Contributions.

- Aynaz Abdollahzadeh - Literature study, Outlier detection Mmodule development, Outlier detection module evaluation, Presentation(Outlier detection), Paper(Outlier detection, Outlier detection performance)
- Afroozeh O. - RegEx Repository, Integration of LLMs and RegEx validation, Outlier detection module evaluation, Presentation (AutoML, RegEx), Paper(Regex syntax validator, Outlier detection performance)
- Dominici L. - Literature study, Presentation, dataset scouting, Paper (Abstract, Introduction, Problem&Approach), Error generator for evaluation.
- Hanjun Liu - Presentation LLM part, Implement LLM & AutoML part in project, Lable data for Type and Regex, Paper(LLM & AutoML evaluation)
- Nguyen M. D. - Presentation (TFDV), Implement metrics, improved TFDV schema inference, combined project pipeline, evaluation with error generator and related sections in the paper

REFERENCES

[1] Syed Anwar. [n.d.]. https://www.kaggle.com/datasets/syedanwarafridi/vehicle-sales-data

[2] Emily Caveness, Paul Suganthan G. C., Zhuo Peng, Neoklis Polyzotis, Sudip Roy, and Martin Zinkevich. 2020. TensorFlow Data Validation: Data Analysis and Validation in Continuous ML Pipelines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, 2793–2796. https://doi.org/10.1145/3318464.3384707

[3] Snehangsu De. [n.d.]. https://www.kaggle.com/datasets/snehangsude/audible-dataset

[4] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. 2020. AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. *arXiv preprint arXiv:2003.06505* (2020).

[5] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG]

[6] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2021. CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. *2021 IEEE 37th International Conference on Data Engineering (ICDE)* (2021), 13–24. https://doi.org/10.1109/ICDE51399.2021.00009

[7] Aleksej Logacjov, Kerstin Bach, Atle Kongsvold, Hilde Bremseth Bårdstu, and Paul Jarle Mork. 2021. HARTH: A Human Activity Recognition Dataset for Machine Learning. *Sensors* 21, 23 (2021). https://doi.org/10.3390/s21237853

[8] Mickaël Mouillé. [n.d.]. https://www.kaggle.com/datasets/kemical/kickstarter-projects

[9] Rashik Rahman. [n.d.]. https://www.kaggle.com/datasets/rashikrahmanpritom/data-science-job-posting-on-glassdoor/

[10] Sergey Redyuk, Zoi Kaoudi, Volker Markl, and Sebastian Schelter. 2021. Automating Data Quality Validation for Dynamic Data Ingestion.. In *EDBT*. 61–72.

[11] Cedric Renggli, Luka Rimanic, Nezihe Merve Gurel, Bojan Karlas, Wentao Wu, and Ce Zhang. 2021. A Data Quality-Driven View of MLOps. *IEEE Data Engineering Bulletin* 44, 1 (2021), 11–23.

[12] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, and Felix Biessmann. 2018. Automating large-scale data quality verification. In *VLDB 2018*.

[13] Shreya Shankar, Labib Fawaz, Karl Gyllstrom, and Aditya Parameswaran. 2023. Automatic and Precise Data Validation for Machine Learning. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM '23)*. Association for Computing Machinery, 2198–2207. https://doi.org/10.1145/3583780.3614786

[14] StackOverflow. [n.d.]. https://survey.stackoverflow.co/2023

[15] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]